

Como Rust e Apache Polars Operam no Naive Bayes

Explicação Didática, Fluxo em Memória e Exemplos Práticos no Dataset de 500.000 Linhas

1. O que são e por que foram escolhidos?

Neste projeto, em vez de recorrer a bancos de dados relacionais lentos ou bibliotecas automáticas de Machine Learning que funcionam como caixas-pretas, usamos a combinação de **Rust** e **Apache Polars** para implementar artesanalmente o classificador:

Tecnologia	O que é de forma simples?	Papel específico no nosso projeto
Rust	Uma linguagem de programação de sistemas ultrarrápida, compilada para binário nativo e que previne falhas de memória em tempo de compilação sem precisar de <i>garbage collector</i> .	Implementa a lógica matemática e as regras de decisão : cálculos de logaritmo natural ($\ln$), suavização de Laplace (+1), soma de log-odds e a função Softmax de conversão para porcentagem.
Apache Polars	Uma biblioteca de DataFrames de altíssimo desempenho escrita em Rust, projetada para substituir ferramentas tradicionais através de processamento colunar vetorizado (Apache Arrow).	Faz o trabalho pesado com os dados : lê o CSV de 500.000 linhas em milissegundos e utiliza todos os núcleos da CPU em paralelo para agrupar e contar as ocorrências de cada classe e feature.

2. Analogia Didática: Como os dois trabalham juntos

Pense no projeto como uma operação em uma grande fazenda de grãos em Mato Grosso:

- **Polars é a Colheitadeira de Alta Potência:** Percorre os 500.000 registros históricos em velocidade máxima, separando e contando as espigas em silos colunares organizados em paralelo.
- **Rust é o Engenheiro Calculista:** Pega as contagens exatas fornecidas pelos silos, aplica as fórmulas de probabilidade condicional de Bayes na ponta do lápis e diz com precisão de porcentagem se o novo talhão vai quebrar ou não.

3. O Fluxo de Execução Passo a Passo (Do CSV à Decisão)

Todo o ciclo é executado em apenas **4 etapas encadeadas** na memória RAM, sem salvar arquivos temporários ou engasgar:

Passo 1: Carga Ultrarrápida e Leitura Preguiçosa (Lazy Execution)

O Polars não carrega strings pesadas uma a uma. Ele mapeia o arquivo `dataset/treinamento_mt_500k.csv` em blocos contíguos de memória estruturados no padrão *Apache Arrow*:

```
// Leitura nativa em Rust via Polars:
let df = CsvReadOptions::default()
    .try_into_reader_with_file_path(Some("dataset/treinamento_mt_500k.csv".into()))?
    .finish()?;
// 500.000 linhas são carregadas em RAM em cerca de ~150 milissegundos!
```

Passo 2: Contagem Paralela Multithread (Priori e Frequências)

O Polars divide o trabalho automaticamente entre todos os núcleos (threads) da máquina usando a biblioteca `Rayon`. Ele executa dois agrupamentos essenciais:

- **Contagem da Priori:** Quantos talhões tiveram `quebra_safra = 'sim'` vs `'nao'` em 500.000 registros.
- **Contagem Condicional:** Quantas vezes cada solo, veranico ou chuva apareceu dentro de cada classe.

```
// Agrupamento paralelo em Rust com Polars:
let contagens = df.lazy()
    .group_by(["classe_pedologica", "quebra_safra"])
    .agg([len().alias("total")])
    .collect()?;
```

Passo 3: A Matemática Artesanal em Rust (bayes.rs)

O Rust pega os números brutos do Polars e aplica a fórmula de Naive Bayes sem bibliotecas prontas.

A) Suavização Dinâmica de Laplace: Para garantir que nenhuma categoria nunca vista zere o cálculo:

$$P(\text{feature} \mid \text{classe}) = (\text{contagem} + 1.0) / (\text{total_classe} + \text{categorias_distintas})$$

B) Log-Probabilidades (Anti-Underflow): Multiplicar decimais pequenos como `0.0031 * 0.042` gera perda de precisão numérica no computador. O Rust transforma produtos em somas simples:

```
// Em vez de: score = p1 * p2 * p3 (gera underflow!)
// O Rust faz:
let mut score_sim = ln_prior_sim;
score_sim += p_solo_sim.ln();
score_sim += p_veranico_sim.ln();
score_sim += p_chuva_sim.ln();
```

Passo 4: Softmax Bayesiano e Recomendação de Campo

Para exibir o resultado de forma intuitiva (ex: 94.6% de risco de quebra), o Rust desfaz a escala logarítmica usando `.exp()` com proteção contra estouro de memória (*overflow*):

```
let max_score = score_sim.max(score_ao);
let exp_sim = (score_sim - max_score).exp();
let exp_ao = (score_ao - max_score).exp();

let prob_quebra = (exp_sim / (exp_sim + exp_ao)) * 100.0;
```

4. Exemplo Prático de Saída no Terminal

Ao compilar e executar o projeto no terminal (`cargo run --release`), o resultado é gerado em menos de meio segundo:

```
$ cargo run --release
  Compiling naive_bayes_agro_mt v0.1.0
  Finished release [optimized] target(s) in 2.14s
  Running `target/release/naive_bayes_agro_mt`

[1/3] Carregando dataset/treinamento_mt_500k.csv via Apache Polars...
  • 500.000 registros carregados em memória em 142ms.
[2/3] Agrupamentos paralelos e Laplace calculados em 168ms.
  • Priori: P(sim) = 33.20% | P(nao) = 66.80%
[3/3] Avaliando os 5 Casos de Teste em Mato Grosso:

--- CASO 1: Baixo Risco - Chapadao dos Parecis ---
Perfil: Latossolo Argiloso | Sem Deficit Reprodutivo | Pivo Central
Probabilidade de Quebra: 2.14% | Safra Regular: 97.86%
Decisao: BAIXO RISCO - Conducao normal da lavoura.

--- CASO 2: Alto Risco Extremo - Solo Arenoso Degradado ---
Perfil: Neossolo Arenoso | Seca Severa R1-R5 | Plantio Dezembro
Probabilidade de Quebra: 96.82% | Safra Regular: 3.18%
Decisao: ALTO RISCO - Acionar seguro rural e travar contratos.

--- CASO 5: Teste de Laplace com Categoria Nao Vista ---
Perfil: Gleissolo Desconhecido (Nao existia na base de 500k!)
Probabilidade de Quebra: 14.30% | Safra Regular: 85.70%
Status:Codigo executou com sucesso (sem divisao por zero via Laplace).

Tempo Total de Execucao: 0.36 segundos!
```

5. Resumo das Vantagens para Apresentação

- **Desempenho Real:** Processa 500.000 linhas e resolve todos os casos em menos de **0,4 segundo**.
- **Transparência Científica:** Nenhuma função pronta de biblioteca de IA; todas as equações foram escritas manualmente no arquivo `bayes.rs`.
- **Inovação Tecnológica:** Utiliza o ecossistema de ponta da engenharia de dados moderna (Rust + Apache Polars + Arrow colunar).